



TRIMESTER OCTOBER/NOVEMBER 2024

TDB6113 DATABASE SYSTEMS

PROJECT PREPARED BY

ABDALLA NADIR ABDALLA	1231302414	AI
ABDELBAGI Mohamed Elbashir	241UT240MA	AI
SHERAZ FAROOQ	241UT241CC	BIA
Abdulsalam Albalawi	1211306264	AI

LAB SECTION: 1F

PRESENTATION VIDEO: <https://youtu.be/ExPrdoRhqkc>

FACULTY OF INFORMATION SCIENCE & TECHNOLOGY
MULTIMEDIA UNIVERSITY
MALAYSIA
27 JANUARY 2025

TABLE OF CONTENTS

1	Introduction.....	3
2	Problem Statements & Objectives	5
3	Business Rules	6
4	Entity Relationship Diagram.....	7
5	Normalization	8
6	Data Dictionary	10
7	SQL/Screenshot	11
8	Business Ideas	20
9	Conclusion	25

1 Introduction

We have created a database for organizations recruiting applicants and after discussing amongst our crew we decided to go with JobApplicationSystem. We collected information from several organizations and assessed what the problems were and fixed accordingly with each category. The database tracks job applicants, their documents submitted, jobs offered, and the current status of the job. This sensitive information is stored in the database and can be overseen by the team with the help of attributes or tables that are defined in accordance.

Interview

The organization is going to be conducting several interviews. It is an important part of the job application system as it contains the relevant information about the interview, who the interviewer or the applicant was, what time and date it was held on, and what job the applicant will be offered. All interviews are tracked within the system. Each of the interviews can be uniquely identified with an Interview_ID. This Interview_ID is linked with a specific applicant (Applicant_ID), job (Job_ID), and interviewer (Interviewer_ID). To facilitate data search, we made Interview_ID the Primary Key.

Interviewer

The job application interviewer is going to be needed, someone who is responsible for recruiting people and can judge who is applicable to join the organization. The interviewer's details are also stored in the Interviewer entity which contains the personal information such as the name and ID of the interviewer. The interviewer also is a part of a department in the organization and can be identifiable by the department attribute. The interview information is kept track of by storing the following: Interview_ID, Interviewer_ID, Applicant_ID, Job_ID, Time, and Date.

Applicant

The applicant is going to be a person who is applying to get a job in the organization, typically submitting their documents, and being interviewed for the position. The applicant's relevant

information for the job is stored. This contains their personal information regarding the interview with their Applicant_ID, their name (Applicant_Name), and their contact information (Contact_Info). The applicant's information is extremely crucial as the organization needs to know or inform the applicant about the job.

Documents

When the applicant submits their documents for the job application they need to be recorded. These documents also need to be retrieved from the right person as a mix up may lead to problems. This is ensured by having the applicant's ID and having the Documents_ID as a Primary Key. For precise measure submission date and type are also added. The submission date will let them know when it was submitted and the type which could include resume, cover letter, courses taken, or personal documents.

Job

The organization is going to be able to know what their jobs are being offered or interviewed. There are a lot of jobs in one organization and exclusively identifying them will be difficult. The job entity is crucial as the superiors will need to verify which job is or was offered. The database includes the Job_ID which can be uniquely identified from the Interview and Job Offer.

Job Offer

When the interview has been completed of the applicant. The organization is going to review the application and uniquely locate which applicant it belongs to. After a rigorous checking process the job application status is updated and for precise identification the date is also included. This makes it easy for everyone in the organization to view the applicant status if it needs to be reviewed by someone. The organization can know how many have applied or were rejected and what jobs have been filled.

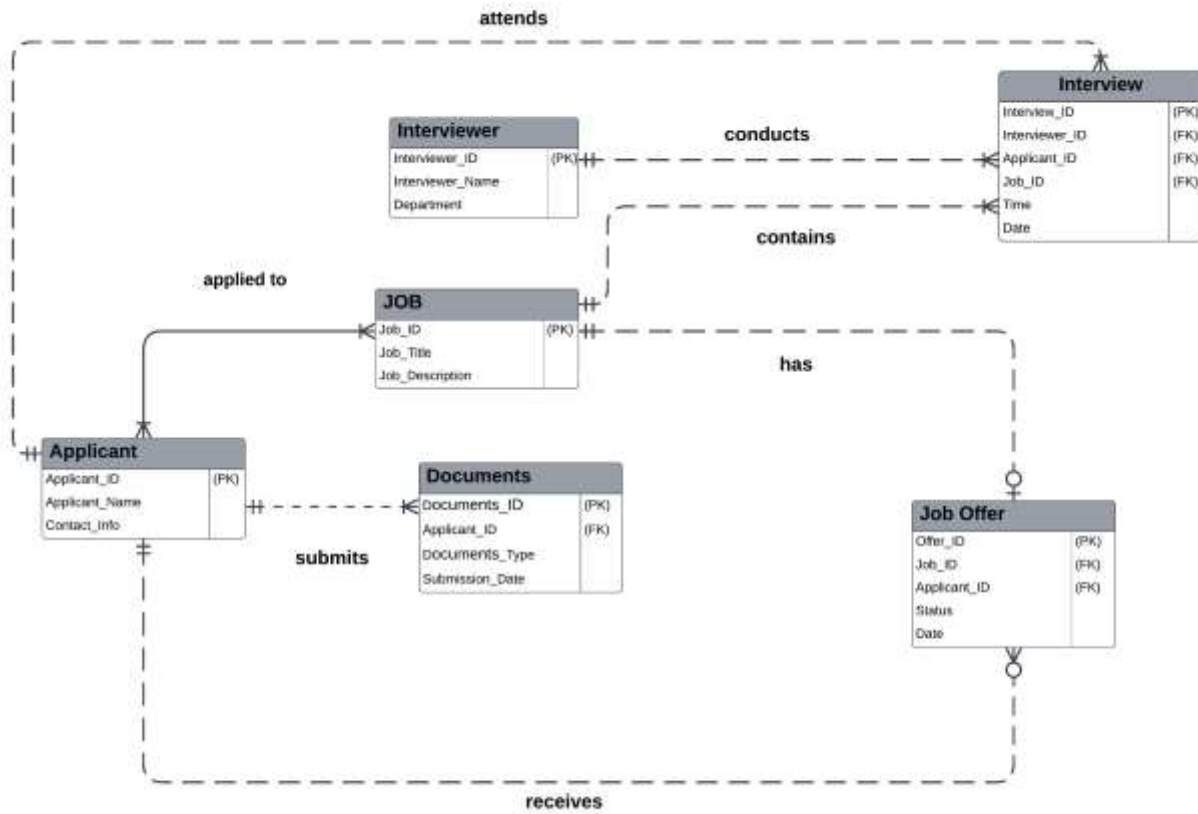
2 Problem Statements & Objectives

Problem Statements	Objectives
There are no existing databases which can organize job applicants with their vital documents and what jobs were offered. Moreover, there are no databases that can link who the interviewer was and when it took place.	Create a database system that organizes job applicants with their documents, tracks job offers, and links interviewers to specific interviews with accurate scheduling.
Organizations get a lot of job applicants and have several departments which require employees. This information gets clustered with data and makes it troublesome for HR teams to communicate and keep track of job applicants accordingly.	Develop a system that categorizes applicants by department. This will make it easier for HR teams to track, manage, and communicate effectively with the job applications.
Tracking applicants with their submitted document is hard for organizations to retrieve from. Scheduling interviews and keeping track of job offers. These interviews may get cancelled due to some circumstances or can end up having double bookings due to miscommunications.	Develop a system that makes it easier to retrieve documents, ensures accurate interview scheduling, and prevents double bookings or cancellations due to miscommunication.
Not having a structured format of stored data will lead to inconsistencies. Duplication of data makes it challenging to keep accurate records of job applicants.	Design a structured database to keep data integrity, prevent duplication, and ensure accurate records of job applicants.

3 Business Rules

1. Every job has many applicants. Each applicant can apply for many jobs.
2. Every job has multiple interviews. Each interview is only related to one job.
3. Every interviewer has many interviews. Each interview is assigned to only one interviewer.
4. Every applicant has multiple interviews. Each interview is only related to one applicant.
5. Applicants can submit multiple documents (e.g., resumes, cover letters). Each document is only linked to one applicant.
6. Every applicant may get many jobs offers. Each job offer belongs only to one applicant.
7. Every job may result in one offer. Each offer is only related to one job.

4 Entity Relationship Diagram



5 Normalization

There are 3 normalization processes. In the first normalization process, there is one entity, i.e. Interview.

Interview (**Interview_ID**, **Interviewer_ID**, **Job_ID**, **Applicant_ID**, Applicant_Name, Contact_Info, Job_Title, Job_Description1, Job_Description2, Job_Description3, Offer_ID, Status, Interviewer_Name, Department1, Department2, Department3, Time, Date, Documents_ID, Documents_Type, Submission_Date)

During 1st normalization process, we found the following partial dependencies and transitive dependencies in table Interview.

The partial dependencies are:

- The (Applicant_Name, Contact_Info) are dependent to (Interview_ID, Interviewer_ID, Job_ID, Applicant_ID)
- The (Job_Title, Job_Description1, Job_Description2, Job_Description3) are dependent to (Interview_ID, Interviewer_ID, Job_ID)
- The (Interviewer_Name, Department1, Department2, Department3) are dependent to (Interview_ID, Interviewer_ID, Job_ID)
- The (Offer_ID, Status) are dependent to (Interview_ID, Interviewer_ID, Job_ID, Applicant_ID)
- The (Time, Date) are dependent to (Interview_ID, Interviewer_ID, Job_ID)
- The (Documents_ID, Documents_Type, Submission_Date) are dependent to (Interview_ID, Applicant_ID)

The transitive dependency is:

The (Interviewer_Name) is dependent to (Interview_ID, Interviewer_ID, Job_ID)
(this is done through Department1, Department2, Department3)

As such, in 2nd normalization process, we will have three entities. Necessary foreign keys are included during the 2nd normalization process.

Interview (**Interview ID**, Interviewer_ID(FK), Job_ID(FK), Applicant_ID(FK), Time, Date)

Interviewer (**Interviewer ID**, Interviewer_Name, Department)

Job (**Job ID**, Job_Title, Job_Description)

Job Offer (**Offer ID**, Job_ID, Applicant_ID, Status, Date)

Applicant (**Applicant ID**, Applicant_Name, Contact_Info)

Documents (**Documents ID**, Applicant_ID, Documents_Type, Submission_Date)

6 Data Dictionary

TABLE	ATTRIBUTE NAME	DESCRIPTION	DATA TYPE	FORMAT	REQUIRED	PK/ FK	REFERENCED TABLE
USER	User_ID	Unique identifier for each user	Varchar(20)	999999999	Yes	PK	-
-	User_Name	Full name of the user	Varchar(60)	Xxxxxx	Yes	-	-
-	User_Email	Email address of the user	Varchar(50)	Xxxxxx@xx.xx	Yes	-	-
-	User_Type	Indicates if the user is a job seeker (J) or employer (E)	Char(1)	J/E	Yes	-	-
JOB	Job_ID	Unique identifier for each job	Varchar(15)	9999-XX	Yes	PK	-
-	Job_Title	Title of the job	Varchar(50)	Xxxxxx	Yes	-	-
-	Job_Description	Description of the job	Varchar(200)	Xxxxxx	No	-	-
-	Job_Location	Location of the job	Varchar(100)	Xxxxxx	No	-	-
APPLICATION	Application_ID	Unique identifier for each application	Varchar(20)	9999999	Yes	PK	-
-	Job_ID	Identifier of the job applied for	Varchar(15)	9999-XX	Yes	FK	JOB
-	User_ID	Identifier of the applicant	Varchar(20)	999999999	Yes	FK	USER
-	Application_Status	Status of the application (Pending, Approved, Rejected)	Varchar(15)	Pending/Approved/Rejected	No	-	-

7 SQL/Screenshot

(a) Database creation:

The following queries initialize the creation of the database which keeps the records. It includes the CREATE and INSERT SQL commands. By having these commands, the first objective, i.e., to develop a database system, is fulfilled.

```
CREATE DATABASE JobApplicationSystem;
```

```
MariaDB [(none)]> CREATE DATABASE JobApplicationSystem;  
Query OK, 1 row affected (0.002 sec)
```

```
USE JobApplicationSystem;
```

```
MariaDB [(none)]> USE JobApplicationSystem;  
Database changed  
MariaDB [JobApplicationSystem]> |
```

```
CREATE TABLE Applicant (  
    Applicant_ID INT AUTO_INCREMENT PRIMARY KEY,  
    Applicant_Name VARCHAR(100) NOT NULL,  
    Contact_Info VARCHAR(100) NOT NULL  
) ENGINE = InnoDB;
```

```
Database changed  
MariaDB [JobApplicationSystem]> CREATE TABLE Applicant (  
    -> Applicant_ID INT AUTO_INCREMENT PRIMARY KEY,  
    -> Applicant_Name VARCHAR(100) NOT NULL,  
    -> Contact_Info VARCHAR(100) NOT NULL  
    -> ) ENGINE = InnoDB;  
Query OK, 0 rows affected (0.024 sec)
```

```
CREATE TABLE JOB (
```

```
Job_ID INT AUTO_INCREMENT PRIMARY KEY,  
Job_Title VARCHAR(100) NOT NULL,  
Job_Description TEXT  
) ENGINE = InnoDB;
```

```
MariaDB [JobApplicationSystem]> CREATE TABLE JOB (  
-> Job_ID INT AUTO_INCREMENT PRIMARY KEY,  
-> Job_Title VARCHAR(100) NOT NULL,  
-> Job_Description TEXT  
-> ) ENGINE = InnoDB;  
Query OK, 0 rows affected (0.007 sec)
```

```
CREATE TABLE Documents (  
Documents_ID INT AUTO_INCREMENT PRIMARY KEY,  
Applicant_ID INT NOT NULL,  
Type VARCHAR(50) NOT NULL,  
Content TEXT NOT NULL,  
FOREIGN KEY (Applicant_ID) REFERENCES Applicant(Applicant_ID)  
) ENGINE = InnoDB;
```

```
MariaDB [JobApplicationSystem]> CREATE TABLE Documents (  
-> Documents_ID INT AUTO_INCREMENT PRIMARY KEY,  
-> Applicant_ID INT NOT NULL,  
-> Type VARCHAR(50) NOT NULL,  
-> Content TEXT NOT NULL,  
-> FOREIGN KEY (Applicant_ID) REFERENCES Applicant(Applicant_ID)  
-> ) ENGINE = InnoDB;  
Query OK, 0 rows affected (0.011 sec)
```

```
CREATE TABLE Job_Offer (  
Offer_ID INT AUTO_INCREMENT PRIMARY KEY,  
Job_ID INT NOT NULL,  
Applicant_ID INT NOT NULL,  
Status ENUM('Pending', 'Accepted', 'Rejected') DEFAULT 'Pending',  
Date DATE NOT NULL,  
FOREIGN KEY (Job_ID) REFERENCES JOB(Job_ID),  
FOREIGN KEY (Applicant_ID) REFERENCES Applicant(Applicant_ID)  
) ENGINE = InnoDB;
```

```

MariaDB [JobApplicationSystem]> CREATE TABLE Job_Offer (
->     Offer_ID INT AUTO_INCREMENT PRIMARY KEY,
->     Job_ID INT NOT NULL,
->     Applicant_ID INT NOT NULL,
->     Status ENUM('Pending', 'Accepted', 'Rejected') DEFAULT 'Pending',
->     Date DATE NOT NULL,
->     FOREIGN KEY (Job_ID) REFERENCES JOB(Job_ID),
->     FOREIGN KEY (Applicant_ID) REFERENCES Applicant(Applicant_ID)
-> ) ENGINE = InnoDB;
Query OK, 0 rows affected (0.016 sec)

```

```

CREATE TABLE Interview (
    Interview_ID INT AUTO_INCREMENT PRIMARY KEY,
    Interviewer_ID INT NOT NULL,
    Job_ID INT NOT NULL,
    Time TIME NOT NULL,
    Date DATE NOT NULL,
    FOREIGN KEY (Job_ID) REFERENCES JOB(Job_ID)
) ENGINE = InnoDB;

```

```

MariaDB [JobApplicationSystem]> CREATE TABLE Interview (
->     Interview_ID INT AUTO_INCREMENT PRIMARY KEY,
->     Interviewer_ID INT NOT NULL,
->     Job_ID INT NOT NULL,
->     Time TIME NOT NULL,
->     Date DATE NOT NULL,
->     FOREIGN KEY (Job_ID) REFERENCES JOB(Job_ID)
-> ) ENGINE = InnoDB;
Query OK, 0 rows affected (0.016 sec)

```

(b). DML Statements (Data Manipulation Language):

1.insert:

```
INSERT INTO Applicant (Applicant_Name, Contact_Info)
VALUES
('John Doe', 'john.doe@example.com'),
('Jane Smith', 'jane.smith@example.com');
```

```
MariaDB [JobApplicationSystem]> INSERT INTO Applicant (Applicant_Name, Contact_Info)
-> VALUES
-> ('John Doe', 'john.doe@example.com'),
-> ('Jane Smith', 'jane.smith@example.com');
Query OK, 2 rows affected (0.012 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

```
INSERT INTO JOB (Job_Title, Job_Description)
VALUES
('Software Engineer', 'Develop and maintain software applications.'),
('Data Analyst', 'Analyze data and generate insights.');
```

```
MariaDB [JobApplicationSystem]> INSERT INTO JOB (Job_Title, Job_Description)
-> VALUES
-> ('Software Engineer', 'Develop and maintain software applications.'),
-> ('Data Analyst', 'Analyze data and generate insights.');
```

```
Query OK, 2 rows affected (0.008 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

```
INSERT INTO Documents (Applicant_ID, Type, Content)
VALUES
(1, 'Resume', 'John Doe Resume Content'),
(2, 'Cover Letter', 'Jane Smith Cover Letter Content');
```

```
MariaDB [JobApplicationSystem]> INSERT INTO Documents (Applicant_ID, Type, Content)
-> VALUES
-> (1, 'Resume', 'John Doe Resume Content'),
-> (2, 'Cover Letter', 'Jane Smith Cover Letter Content');
```

```
Query OK, 2 rows affected (0.003 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

```
INSERT INTO Job_Offer (Job_ID, Applicant_ID, Status, Date)
VALUES
(1, 1, 'Pending', '2023-10-15'),
(2, 2, 'Accepted', '2023-10-20');
```

```
MariaDB [JobApplicationSystem]> INSERT INTO Job_Offer (Job_ID, Applicant_ID, Status, Date)
-> VALUES
-> (1, 1, 'Pending', '2023-10-15'),
-> (2, 2, 'Accepted', '2023-10-20');
Query OK, 2 rows affected (0.004 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

```
INSERT INTO Interview (Interviewer_ID, Job_ID, Time, Date)
VALUES
(101, 1, '10:00:00', '2023-10-10'),
(102, 2, '14:00:00', '2023-10-12');
```

```
MariaDB [JobApplicationSystem]> INSERT INTO Interview (Interviewer_ID, Job_ID, Time, Date)
-> VALUES
-> (101, 1, '10:00:00', '2023-10-10'),
-> (102, 2, '14:00:00', '2023-10-12');
Query OK, 2 rows affected (0.002 sec)
Records: 2 Duplicates: 0 Warnings: 0
```

2. Trigger:

Trigger to Notify When Job Offer Status Changes:

DELIMITER //

CREATE TRIGGER UpdateJobOfferStatusNotification

AFTER UPDATE ON Job_Offer

FOR EACH ROW

BEGIN

 DECLARE NotificationMessage TEXT;

 IF NEW.Status <> OLD.Status THEN

 SET NotificationMessage = CONCAT('Your job offer for Job ID ', NEW.Job_ID, ' is now ',
NEW.Status);

 INSERT INTO Documents (Applicant_ID, Type, Content)

 VALUES (NEW.Applicant_ID, 'Notification', NotificationMessage);

 END IF;

END //

DELIMITER ;

```

MariaDB [JobApplicationSystem]> CREATE TRIGGER UpdateJobOfferStatusNotification
-> AFTER UPDATE ON Job_Offer
-> FOR EACH ROW
-> BEGIN
->   DECLARE NotificationMessage TEXT;
->   IF NEW.Status <> OLD.Status THEN
->     SET NotificationMessage = CONCAT('Your job offer for Job ID ', NEW.Job_ID, ' is now ', NEW.Status);
->     INSERT INTO Documents (Applicant_ID, Type, Content)
->       VALUES (NEW.Applicant_ID, 'Notification', NotificationMessage);
->   END IF;
-> END //
Query OK, 0 rows affected (0.020 sec)

```

DELIMITER //

CREATE FUNCTION CalculateAverageSalary()

RETURNS DECIMAL(10, 2)

DETERMINISTIC

BEGIN

DECLARE avg_salary DECIMAL(10, 2);

SELECT AVG(Salary) INTO avg_salary FROM JOB;

RETURN avg_salary;

END //

DELIMITER ;

```

MariaDB [JobApplicationSystem]> DELIMITER //
MariaDB [JobApplicationSystem]>
MariaDB [JobApplicationSystem]> CREATE FUNCTION CalculateAverageSalary()
-> RETURNS DECIMAL(10, 2)
-> DETERMINISTIC
-> BEGIN
->   DECLARE avg_salary DECIMAL(10, 2);
->   SELECT AVG(Salary) INTO avg_salary FROM JOB;
->   RETURN avg_salary;
-> END //
Query OK, 0 rows affected (0.016 sec)

```

```

MariaDB [JobApplicationSystem]>

```

```

MariaDB [JobApplicationSystem]> DELIMITER ;|

```

3. Stored Procedure:

Procedure to Get All Job Offers for an Applicant:


```

MariaDB [JobApplicationSystem]> CREATE PROCEDURE GetJobOffersForApplicant(IN ApplicantID INT)
-> BEGIN
->     SELECT
->         Job_Offer.Offer_ID,
->         JOB.Job_Title,
->         Job_Offer.Status,
->         Job_Offer.Date
->     FROM Job_Offer
->     JOIN JOB ON Job_Offer.Job_ID = JOB.Job_ID
->     WHERE Job_Offer.Applicant_ID = ApplicantID;
-> END //
Query OK, 0 rows affected (0.019 sec)

```

Call the Procedure:

CALL GetJobOffersForApplicant(1);

```

MariaDB [JobApplicationSystem]> CALL GetJobOffersForApplicant(1);
+-----+-----+-----+-----+
| Offer_ID | Job_Title          | Status | Date       |
+-----+-----+-----+-----+
|          1 | Software Engineer | Pending | 2023-10-15 |
+-----+-----+-----+-----+
1 row in set (0.013 sec)

Query OK, 0 rows affected (0.015 sec)

```

DELIMITER //

CREATE PROCEDURE GetJobsByLocation(IN location_name VARCHAR(100))

BEGIN

SELECT

Job_ID,

Job_Title,

Job_Description,

Salary

FROM JOB

WHERE Location = location_name;

END //

DELIMITER ;

```

MariaDB [JobApplicationSystem]> CREATE PROCEDURE GetJobsByLocation(IN location_name VARCHAR(100))
-> BEGIN
->     SELECT
->         Job_ID,
->         Job_Title,
->         Job_Description,
->         Salary
->     FROM JOB
->     WHERE Location = location_name;
-> END //
Query OK, 0 rows affected (0.006 sec)

```

4. Function:

Function to Count Job Offers for a Job:

DELIMITER //

CREATE FUNCTION CountJobOffersForJob(JobID INT)

RETURNS INT

DETERMINISTIC

BEGIN

 DECLARE OfferCount INT;

 SELECT COUNT(*) INTO OfferCount

 FROM Job_Offer

 WHERE Job_Offer.Job_ID = JobID;

 RETURN OfferCount;

END //

DELIMITER ;

```

MariaDB [JobApplicationSystem]>
MariaDB [JobApplicationSystem]> CREATE FUNCTION CountJobOffersForJob(JobID INT)
-> RETURNS INT
-> DETERMINISTIC
-> BEGIN
->     DECLARE OfferCount INT;
->     SELECT COUNT(*) INTO OfferCount
->     FROM Job_Offer
->     WHERE Job_Offer.Job_ID = JobID;
->     RETURN OfferCount;
-> END //
Query OK, 0 rows affected (0.010 sec)

```

Use the Function:

```

MariaDB [JobApplicationSystem]> SELECT CountJobOffersForJob(1) AS OfferCount;
+-----+
| OfferCount |
+-----+
|          1 |
+-----+
1 row in set (0.008 sec)

```

DELIMITER //

```

CREATE FUNCTION CalculateAverageSalary()
RETURNS DECIMAL(10, 2)
DETERMINISTIC
BEGIN
    DECLARE avg_salary DECIMAL(10, 2);
    SELECT AVG(Salary) INTO avg_salary FROM JOB;
    RETURN avg_salary;
END //

```

DELIMITER ;

```

Database changed
MariaDB [JobApplicationSystem]> SELECT CalculateAverageSalary() AS AverageSalary;
+-----+
| AverageSalary |
+-----+
|          NULL |
+-----+
1 row in set (0.015 sec)

```

8 Business Ideas

Name: Abdalla nadir abdalla

ID: 1231302414

-How can your system address a real-life problem?

The Job Application System enables job candidates to handle their application submissions and monitor progress as well as access personalized job listings. The system aids employers through powerful screening algorithms and matching filters that make candidate choice simpler.

-Who are the potential users of your system and what specific needs do they have?

Our potential users are job seekers and employees, and they want an easy application process that lets users track their status alongside personalized job recommendations. The system also helps filter candidates and manage applications effectively.

-How will you promote your system and reach potential users?

Throughout Digital Marketing, the use of social media, search engine optimization (SEO), and targeted ads to reach job seekers and employers. Partnerships, Collaborate with job boards, universities, and recruitment agencies. Referral Programs: Users who refer others receive incentives as part of our referral program.

-What are the strengths and weaknesses of your system? You may consider conducting a SWOT analysis of your system

Strengths: The platform simplifies job applications because it provides an interface that both candidates and organizations can use easily while its advanced algorithm makes candidate-job matching more likely.

Weaknesses: The company needs to handle sensitive personal and professional information because of data privacy concerns as well as confront market entry limitations due to numerous existing job platforms.

-What are the potential challenges (e.g., cost, technical requirements and issues, etc.) that could impact the successful implementation of your system in addressing the identified real-life problem?

The system's total expenses along with maintenance fees and technical requirements for infrastructure present challenges for large data and user volumes while user adaptation and regulatory compliance requirements must be met.

Name: Abdulsalam Albalawi

ID: 1211306264

Problem Identification and Solution Alignment

The problem

is that job hunting is not easy because sometimes a seeker applies for a job that is irrelevant, and the employer wastes time looking through irrelevant applications.

We can make a smart database for a Job Application

System which is capable of matching job-seekers to job openings based on their preferences and skills; it enables employers to find the right person in time. It updates the users with the status of applications besides updating the hiring jobs.

Strategic Planning

Who is going to use this system, and what do they need?

1. Job Seekers:

They Need: job recommendations based on qualifications.

They Want: ease in application for a job and to trace the progress from it.

2. Employers:

They Require appropriate candidates on time, which will reduce time and costs, respectively.

b. How to Promote It?

1. Promotions will be made on LinkedIn and Instagram.

2. Partners such as universities and job fairs will be needed to attract students and professionals, respectively.

Implementation Issues

a. Strengths and Weaknesses: SWOT

Strengths:

Matches people to jobs or candidates quickly and an accurate manner.

It can be developed to handle numerous users.

It Offers valuable insight into hiring trends as well as skill gaps.

Disadvantages:

It's expensive to develop and make services.

It takes considerable time to have enough user data to function decently.

b. Challenges and Solution

1. Expensive to develop enhanced features.

Solution: it can do so gradually, say, start with simple facilities and then introduce more higher level features in due course.

2.

Data security of the users.

Solution: taking hard-line measures in terms of security implications and privacy policies.

3. Inducing people to use it.

Solution: free trials or concessions on prices and promotions through partnerships.

Name: ABDELBAGI Mohamed Elbashir

ID: 241UT240MA

Our system offers innovative ways for both employers and job applicants to deal with inefficiencies in the hiring process. The lengthy and inaccurate process of matching people with opportunities typically irritates people. Our system makes job searches easier by expediting the application and screening process, pointing recruiters to the best applicants, and offering job applicants suggestions to fill skills gaps. This strategy lowers ineffective and encourages better hiring results.

To promote our system effectively, we can employ diverse strategies. Social media platforms like Instagram, TikTok, and LinkedIn offer chances to produce informative material that illustrates how our platform functions. We can successfully get a potential user by sharing success stories, providing job-hunting advice, and highlighting the advantages of the system. Also offering a free trial that will enable users to witness the platform's worth directly. A strong promotional impact is further ensured by conducting surveys to identify weak points in current systems and adjusting our approach accordingly.

Evaluating our system's advantages and disadvantages is essential. Both recruiters and job applicants save time thanks to the user-friendly interface, which increases efficiency. While recruitment agencies gain from advanced filtering tools and organized communication, job applicants can use smooth job matching and application tracking. Nevertheless, there are drawbacks, such as the requirement for active user registration and upfront development expenses. However, as we continue to improve the platform, these are being addressed.

The increasing need for digital recruitment platforms presents numerous opportunities, putting our system in a position to serve a tech-savvy user base. However, there are risks, such as rivalry from well-known websites like Glassdoor and LinkedIn. We aim to establish a position in this cutthroat market by emphasizing distinctive features and outstanding user experience.

Problems with costs and technical requirements may hinder implementation. Development and hosting require substantial funding, and it's crucial to ensure that the system can expand to accommodate a growing user base. Concerns regarding customer resistance to leaving popular platforms are still present. To overcome this, a clear description of our approach's benefits and evidence of its effectiveness are req

Name: SHERAZ FAROOQ

ID: 241UT241CC

How can your system solve a real-life problem?

Our system can solve the tedious problem of tracking which applicants have applied with the Applicant_ID and retrieve what type of documents were submitted from Document_Type.

How will you advertise your system?

We will be advertising our systems to new organizations and to organizations which are having trouble with saving data of their job applications. These advertisements include creating posters and videos sharing them to our social media platforms which will attract people who either work in organizations or organizations having trouble in keeping data.

What are the strengths and weaknesses of your system?

Our strengths are that we have an easy-to-use database which can organize job applications. Our database has the ability to get what types of documents were submitted and the status of job applications. Our weaknesses are that unfortunately our database does not have some details of applicants such as what university or school they have studied at and what each individuals' genders are.

Who are the potential users of your system and what specific needs do they have?

Our potential users are organizations that are recruiting new employees. Organizations require to know when the interview was conducted, who the applicant is, and who the interviewer was. This will be essential as the organization needs to be able to identify who is responsible for recruiting the applicant. One more important information that is necessary is the status of the application as they should know that only the required number of employees can be recruited.

What are the potential challenges (e.g., cost, technical requirements and issues, etc.) that could impact the successful implementation of your system in addressing the identified real-life problem?

We will be requiring a server in order to solve these problems. Researching from some sources we have identified that Microsoft charges \$0.6 per hour for an SQL server which has two core and 10.2GB of RAM. There is a need of an expert of database system on site. This can be provided by us or hired by the organization by a third party.

9 Conclusion

First Part Conclusion:

The database is well designed, following the principles of normalization, scalability, and automation of processes. One can notice an effective usage of a primary key, foreign key, and constraints that assure integrity within the data, along with flexibility within it. Its weaknesses include exception handling that is exceptionally minimal; bottlenecks regarding performance may be observed; and security risks like no encryption and no user permissions. Furthermore, audit trails or business rules are managed within the application and not the database.

Second Part Conclusion:

This paper, therefore, proposes a system that will, ideally, eradicate the present inefficiencies related to maintaining records of applicants for a vacant position, scheduling interviews, and following up on job offers. By proposing a structured database, it cuts down duplication, miscommunication, and unorganized storing of data. It streamlines human resource functions in categorizing applicants and interviews coupled with interviewers so that no discrepancies in scheduling take place in regard to data integrity and operational efficiency.

Future Goals:

The future targets of the system are to enhance accuracy, reduce errors, which will in turn make the database more efficient and ready for future growth. Secondly, it also provides security for sensitive information and access to only authorized personnel; this would facilitate ease in document management and scheduling. It will also provide various tools in facilitating monitoring changes more readily, making effective decisions and enhancements in reporting capabilities. The design will be done in such a way that it remains flexible, accommodating future organizational needs and expansion.